

OpenPair

Intelligent AI Routing Engine

System Requirement Document (SRD)

Version	1.0 — Draft
Status	In Progress
Last Updated	May 2025

1. Introduction

1.1 Purpose

จุดประสงค์ของโปรเจกต์ "OpenPair" คือการสร้างระบบ AI Router ที่มีความแม่นยำสูง เพื่อทำหน้าที่เป็น "จราจรทางอากาศ" (Intelligence Layer) ให้กับข้อมูล ระบบจะทำการจำแนกงาน (Task) ที่เข้ามา แล้วเลือกใช้ AI Model หรือ Ecosystem ที่เหมาะสมที่สุดโดยอัตโนมัติ เป้าหมายหลักของระบบ ได้แก่:

- **Cost Optimization** — ลดค่าใช้จ่ายจากการเรียกใช้ Model ระดับสูงฟรีๆ (ประหยัดได้สูงสุดถึง 85%)
- **Speed Improvement** — เพิ่มความเร็วในการตอบสนองโดยกระจายงานง่ายๆ ไปยัง Model ขนาดเล็กที่ทำงานได้เร็วกว่า
- **Quality Preservation** — ยังคงคุณภาพของผลลัพธ์ไว้ด้วยระบบ Fallback และ LLM-as-a-Judge อัตโนมัติ
- **Developer Experience** — ติดตั้งและใช้งานได้ง่ายผ่าน pip install และ CLI Interface

1.2 Scope

ระบบ OpenPair จะถูกพัฒนาในรูปแบบ Hybrid Architecture โดยมีองค์ประกอบหลักดังนี้:

- **Core Engine:** พัฒนาด้วยภาษาประสิทธิภาพสูง (Rust หรือ Go) เพื่อความเร็วระดับมิลลิวินาที ไม่มี Garbage Collector Overhead
- **Python Binding:** ใช้ PyO3 + Maturin (สำหรับ Rust) หรือ cgo (สำหรับ Go) เพื่อห่อหุ้มเป็น Python Package
- **CLI Interface:** รองรับการใช้งานผ่าน Terminal สำหรับนักพัฒนา
- **Python Library:** สามารถ import ใช้ใน codebase ได้ทันที ผ่าน pip install openpair
- **Web Application (Future):** รองรับการขยายเป็น REST API และ Web Dashboard ในอนาคต

1.3 Definitions & Glossary

Term	Definition
AI Router	ระบบที่จำแนกและส่งต่องานไปยัง AI Model ที่เหมาะสมโดยอัตโนมัติ
Task Classifier	โมดูลที่วิเคราะห์ Intent และ Complexity ของ Prompt ที่รับเข้ามา
Model Registry	ฐานข้อมูล In-Memory ที่เก็บ Capabilities และ Constraints ของแต่ละ Model
LLM Cascade	กลไก Fallback ที่ส่งงานต่อไปยัง Model ที่เก่งกว่าเมื่อผลลัพธ์ไม่ผ่านเกณฑ์
Semantic Router	การจำแนกงานโดยใช้ Vector Embeddings และ Cosine Similarity
Complexity Score	คะแนน 1-10 ที่ระบบใช้ตัดสินใจเลือก Model (สูง = ใช้ Model ดี)
PyO3 / Maturin	เครื่องมือสำหรับสร้าง Python Bindings จาก Rust Code
FrugalGPT	งานวิจัยจาก Stanford ที่เสนอแนวทางลดต้นทุน LLM โดยใช้ Routing

Term	Definition
RouteLLM	Framework Open-Source สำหรับบริการ Route ระหว่าง Strong/Weak Model

2. General Description

2.1 User Characteristics

ผู้ใช้งานหลักของระบบ OpenPair แบ่งออกเป็น 3 กลุ่ม:

User Type	Role / Description	Primary Goal
AI Engineer	นักพัฒนาที่สร้าง AI Pipeline และ LLM Applications	ต้องการ Routing Logic ที่ยืดหยุ่น ผสานกับ codebase ได้ง่าย
Software Developer	Dev ทั่วไปที่ต้องการเรียกใช้ AI ใน Application ของตน	ต้องการ API ที่ simple และ cost-effective
Head of Product / CTO	ผู้บริหารที่ต้องการ Dashboard ดูต้นทุนและประสิทธิภาพ	ต้องการลดค่า API สูงสุด 85% พร้อม Analytics

2.2 Product Perspective

OpenPair ทำงานในฐานะ Middleware Layer ระหว่าง Application ของผู้ใช้กับ AI Model ต่างๆ:

Layer	Technology	Responsibility
Frontend / CLI	Python 3.10+, Click/Typer	รับ Input จากผู้ใช้, แสดงผลลัพธ์, จัดการ Config
Python API Layer	Python Library (pip)	Public API สำหรับ Developers, Type Hints, Async Support
Binding Layer	PyO3 + Maturin (Rust) หรือ cgo (Go)	สะพานเชื่อมระหว่าง Python และ Core Engine
Core Engine	Rust หรือ Go	Classification Logic, Vector Math, Decision Engine
Model Registry	In-Memory HashMap (Rust/Go)	เก็บ Capabilities, Pricing, Context Limits ของทุก Model
External APIs	OpenAI, Anthropic, Google, Ollama	ปลายทางสุดท้ายที่รับ Prompt และส่งคำตอบกลับ

2.3 Supported AI Models (Initial Registry)

Model	Provider	Best For	Context Window
GPT-4o	OpenAI	Reasoning, Code,	128K tokens

Model	Provider	Best For	Context Window
		Vision	
GPT-4o-mini	OpenAI	Fast, Low-Cost Tasks	128K tokens
Claude 3.5 Sonnet	Anthropic	Analysis, Writing, Code	200K tokens
Claude 3 Haiku	Anthropic	Ultra-Fast, Cheap	200K tokens
Gemini 1.5 Pro	Google	Long Context, Multimodal	1M tokens
Gemini 1.5 Flash	Google	Speed, Cost Efficiency	1M tokens
Llama 3.1 70B	Meta (Ollama)	Local / Private Deployment	128K tokens
Mistral 7B	Mistral (Ollama)	Lightweight Routine Tasks	32K tokens

3. Functional Requirements

ระบบทำงานผ่านแนวคิด Unit of Work ที่มองต้นทุน ความเร็ว และคุณภาพเป็นหลัก

3.1 Multi-Stage Classification

3.1.1 Semantic Router

ระบบต้องสามารถแปลง Prompt ที่รับเข้ามาเป็น Vector Embeddings และเทียบความคล้ายคลึงกับกลุ่มงานที่กำหนดไว้ด้วย Cosine Similarity:

- ใช้ BERT-base หรือ Sentence-Transformers สำหรับการสร้าง Embeddings
- กำหนด Route Groups ล่วงหน้า เช่น Code, Summarization, Translation, Creative Writing, Data Analysis, Q&A
- คำนวณ Cosine Similarity ใน Core Engine (Rust/Go) เพื่อความเร็วสูงสุด
- Threshold การจำแนก: ≥ 0.75 = High Confidence, $0.50-0.74$ = Medium, < 0.50 = Fallback to LLM Judge

3.1.2 Intent Classifier (Gatekeeper)

ระบบต้องมี Gatekeeper ที่วิเคราะห์ความซับซ้อนของงาน:

- Complexity Score 1–10: วัดจาก Prompt Length, Vocabulary Diversity, Chain-of-Thought Requirement
- Task Type Detection: Generative vs. Extractive vs. Reasoning
- Context Requirement: ตรวจสอบว่างานต้องการ Long Context ($> 16K$ tokens) หรือไม่
- Sensitivity Check: ตรวจสอบว่างานเกี่ยวข้องกับข้อมูลส่วนตัวหรือ Compliance ที่ต้องการ Local Model

3.2 Model Capabilities Registry

3.2.1 Dynamic Metadata Store

ระบบต้องมีฐานข้อมูล In-Memory ที่อัปเดตได้:

- เก็บข้อมูล: Model Name, Provider, Capabilities Tags, Pricing (per 1K token), Context Window, Average Latency
- รองรับการอัปเดต Registry แบบ Hot-Reload โดยไม่ต้องรีสตาร์ทระบบ
- รองรับ Custom Model เช่น Ollama Local Models ที่ผู้ใช้กำหนดเอง

3.2.2 Constraint Checker

ก่อนส่งงานให้ Model ระบบต้องตรวจสอบ:

- Context Window: ความยาว Prompt ต้องไม่เกิน Context Limit ของ Model

- **API Availability:** ตรวจสอบ Status ของ Provider API (Timeout < 200ms)
- **Rate Limit:** ติดตาม Token Usage ของแต่ละ Model เพื่อหลีกเลี่ยง Rate Limit Error
- **Cost Budget:** หากผู้ใช้กำหนด Budget Cap ระบบจะเลือก Model ที่ถูกกว่าเมื่อใกล้ถึงขีดจำกัด

3.3 Decision Logic Engine

3.3.1 Complexity-Based Routing

Complexity Score	Task Category	Recommended Model Tier	Example Tasks
1 – 3 (Low)	Routine / Simple	Small / Fast (Haiku, Flash, Mistral)	FAQ, Translation, Keyword Extraction
4 – 6 (Medium)	Standard	Mid-tier (GPT-4o-mini, Sonnet)	Summarization, Email Draft, Code Review
7 – 9 (High)	Complex Reasoning	Top-tier (GPT-4o, Claude 3.5 Sonnet)	Architecture Design, Multi-step Reasoning
10 (Critical)	Expert / Long Context	Best Available + Long Context Model	Legal Analysis, 100K+ Token Documents

3.3.2 Cost vs. Performance Framework

นำแนวคิดจากงานวิจัย FrugalGPT และ RouteLLM มาประยุกต์ใช้:

- **Quality Threshold (Q):** ผู้ใช้กำหนดเกณฑ์คุณภาพขั้นต่ำ (0.0 – 1.0) ระบบจะเลือก Model ที่ถูกที่สุดที่ผ่านเกณฑ์
- **Cost Weight (λ):** น้ำหนักที่ให้กับต้นทุนในสูตร $Score = Quality - \lambda \times Cost$
- **Learned Routing:** ระบบบันทึก Feedback Loop เพื่อปรับ Routing Decision ให้ดีขึ้นตามเวลา

3.4 Fallback & LLM-as-a-Judge

3.4.1 Quality Verification

- ระบบใช้ Small LLM (เช่น GPT-4o-mini) ตรวจสอบคุณภาพคำตอบจาก Model หลัก
- **Verification Criteria:** Factual Accuracy, Completeness, Format Compliance
- หากคะแนนต่ำกว่า Threshold ที่กำหนด ระบบจะ Reroute อัตโนมัติ

3.4.2 LLM Cascade

Step	Action	Condition	Next Step
1	ส่งงานให้ Small Model	Complexity Score ≤ 5	รอผลลัพธ์

Step	Action	Condition	Next Step
2	Judge ตรวจสอบผล	ผลได้รับจาก Model	ผ่าน → ส่งกลับ / ไม่ผ่าน → Step 3
3	Reroute ไป Mid Model	Judge Score < Threshold	ส่งซ้ำด้วย Model ดีกว่า
4	Reroute ไป Top Model	Mid Model ยังไม่ผ่าน	ส่งซ้ำด้วย Best Model
5	Return with Warning	Top Model ผลยังไม่ดี	ส่งผลพร้อม Low Confidence Flag

4. Technical Implementation Specifications

4.1 Architecture: Hybrid Core & Python Binding

4.1.1 Core Engine (Rust — Recommended)

เลือกว่าแนะนำ Rust เป็น Core Engine หลัก เนื่องจากข้อได้เปรียบดังนี้:

- Zero-cost abstractions: ไม่มี Runtime Overhead
- No Garbage Collector: ไม่มี Stop-the-World Pause ทำให้ Latency คาดเดาได้
- Memory Safety: Borrow Checker ป้องกัน Memory Leak และ Data Race
- SIMD Support: ใช้ AVX2/AVX-512 สำหรับ Vector Math (Cosine Similarity) ได้โดยตรง
- PyO3 Ecosystem: การสร้าง Python Binding ที่สมบูรณ์และมี Community ขนาดใหญ่

4.1.2 Data Structures (In-Memory)

Structure	Type (Rust)	Usage
Model Registry	HashMap<String, ModelMeta>	เก็บ Capabilities ของทุก Model
Route Groups	Vec<RouteGroup>	เก็บ Intent Categories + Embeddings
Request Queue	VecDeque<RouterRequest>	Queue สำหรับ Async Processing
Decision Cache	LRU Cache<Hash, ModelId>	Cache ผล Routing ที่เคยตัดสินใจแล้ว
Usage Tracker	DashMap<ModelId, UsageStat>	Thread-safe Concurrent Usage Counter

4.2 Data Processing Pipeline

ขั้นตอนการประมวลผลตั้งแต่รับ Input จนถึงส่งผลลัพธ์กลับ:

Step	Layer	Action	Output
1	Python API	รับ Input (Prompt, Config, API Keys)	Validated Request Object
2	Binding Layer	Serialize Python Object → Rust Struct	RouterRequest Struct
3	Core Engine	Tokenize + Embed Prompt (BERT/ST)	Float32 Vector (768-dim)
4	Core Engine	Cosine Similarity vs. Route	Intent Label + Confidence Score

Step	Layer	Action	Output
		Groups	
5	Core Engine	Complexity Scoring Algorithm	Score 1-10 + Task Metadata
6	Core Engine	Constraint Check (Context, Budget)	Eligible Model List
7	Core Engine	Decision: เลือก Optimal Model	ModelId + Routing Reason
8	Python API	ส่ง Request ไป Provider API	LLM Response
9	Judge Module	ตรวจสอบคุณภาพ (Optional)	Quality Score + Flag
10	Python API	ส่งผลลัพธ์ + Metadata กลับ	RouterResponse Object

4.3 Unified API Design

ระบบทำงานเหมือน Proxy Layer — ผู้ใช้ส่ง Prompt มา 1 จุด ระบบกระจายไปยัง Provider ที่เหมาะสมโดยอัตโนมัติ:

API Endpoint / Method	Input Parameters	Output
router.route(prompt)	prompt: str, config: RouterConfig	RouterResponse (model_used, text, cost, latency)
router.route_async(prompt)	prompt: str, config: RouterConfig	Async Generator (Streaming Support)
router.explain(prompt)	prompt: str	RoutingDecision (why, score, alternatives)
router.add_model(meta)	ModelMeta dataclass	Updated Registry Confirmation
router.stats()	None	UsageReport (cost, calls, accuracy per model)

5. Non-Functional Requirements

5.1 Performance

Metric	Target	Notes
Routing Decision Latency	< 5 ms	P99 — ไม่รวม Network Time ไปยัง Provider
Embedding Computation	< 50 ms	สำหรับ Prompt ขนาด 1K tokens
Memory Footprint (Core)	< 100 MB	ขณะ Idle, รวม Model Registry และ Embeddings
Throughput	≥ 1,000 req/sec	บน Hardware 4-core, 8GB RAM
Cache Hit Rate (Target)	≥ 40%	LRU Cache สำหรับ Repeated/Similar Prompts
Cold Start Time	< 500 ms	เวลาโหลด Registry และ Embeddings ครั้งแรก

5.2 Reliability & Availability

- Automatic Failover:** หาก Model ปลายทาง Downtime ระบบต้องสลับไปยัง Model สำรองที่มี Capability ใกล้เคียงภายใน < 200ms
- Circuit Breaker:** หาก Provider มี Error Rate > 10% ใน 60 วินาที ระบบจะหยุดส่งงานชั่วคราวและแจ้งเตือน
- Health Check:** ระบบตรวจสอบ Provider API Status ทุก 30 วินาทีใน Background
- Graceful Degradation:** หาก Core Engine สัมเหลว Python Layer ต้องสามารถ Fallback ไปยัง Direct API Call ได้

5.3 Security

- API Key Management:** ไม่เก็บ API Key ใน Plain Text — ใช้ Environment Variables หรือ Secrets Manager
- No Prompt Logging (Default):** ระบบไม่บันทึกเนื้อหา Prompt ลง Log โดย Default (ป้องกัน Data Leak)
- Local Model Support:** รองรับ Ollama สำหรับ Use Case ที่ต้องการ Data Privacy สูง (ไม่ส่งข้อมูลออกนอก)
- Input Sanitization:** ตรวจสอบ Prompt สำหรับ Prompt Injection Attack ก่อนส่งต่อ

5.4 Maintainability & Extensibility

- Plugin Architecture:** รองรับการเพิ่ม Custom Classifier หรือ Router Strategy ผ่าน Plugin Interface
- Model Agnostic:** ไม่ผูกติดกับ Provider ใด รองรับการเพิ่ม Model ใหม่ผ่าน YAML/JSON Config
- Observability:** รองรับ OpenTelemetry สำหรับ Tracing และ Prometheus สำหรับ Metrics

- **Semantic Versioning:** ทุก Breaking Change ต้องมาพร้อม Major Version Bump และ Migration Guide

5.5 Compatibility

- Python 3.10, 3.11, 3.12
- OS: macOS (ARM/x86), Linux (Ubuntu 20.04+), Windows 10+ (WSL2)
- Architecture: x86_64, ARM64 (Apple Silicon)
- ไม่ต้องการ CUDA/GPU — ออกแบบมาสำหรับ CPU-only Environment

6. Development Roadmap

Phase 1: MVP — Architecture & CLI Foundation

Timeline: Weeks 1–4

Goal: สร้าง Working Prototype ที่ทดสอบได้จริง

Task	Owner	Deliverable
สร้าง Rust Core พร้อม Hello World	Core Dev	Rust project structure
สร้าง Python Binding ด้วย PyO3 + Maturin	Core Dev	pip install . สำเร็จ
สร้าง CLI Interface ด้วย Typer/Click	Python Dev	openpair route "Hello"
Hard-coded Rule-based Router (3 Models)	Core Dev	Routes to GPT/Claude/Gemini
Unit Tests Coverage ≥ 80%	All	pytest + cargo test passing
Docker Image สำหรับ Development	DevOps	docker-compose up ใช้งานได้

Phase 2: Logic & Routing Engine

Timeline: Weeks 5–9

Goal: ระบบตัดสินใจได้อย่างชาญฉลาดผ่าน Rule-based Logic

Task	Owner	Deliverable
Model Registry HashMap + YAML Config	Core Dev	Dynamic model loading
Complexity Scoring Algorithm (Rule-based)	AI Dev	Score 1-10 per prompt
Constraint Checker (Context, Budget)	Core Dev	Block oversized prompts
LLM Cascade + Fallback Mechanism	AI Dev	Auto-retry on poor quality
Basic Logging & Cost Tracking	Python Dev	router.stats() working
Integration Tests vs. Real APIs	QA	E2E test suite passing

Phase 3: Semantic Intelligence

Timeline: Weeks 10–15

Goal: ระบบเข้าใจ Intent ของ Prompt ได้ไม่ хуже Rule-based

Task	Owner	Deliverable
Integrate BERT/Sentence-Transformers	AI Dev	Embedding pipeline working
Cosine Similarity ใน Rust Core (SIMD)	Core Dev	< 5ms classification
Intent Classification Training Dataset	AI Dev	Labeled dataset 5K+ examples
LLM-as-a-Judge Implementation	AI Dev	Quality scoring working
Benchmark vs. Always-GPT4 Baseline	QA	Cost report: X% savings
A/B Testing Framework สำหรับ Routing	AI Dev	Compare routing strategies

Phase 4: Packaging, Analytics & Web App

Timeline: Weeks 16–22

Goal: Production-ready Package + Web Dashboard

Task	Owner	Deliverable
Publish to PyPI (openpair)	DevOps	pip install openpair
REST API Server (FastAPI)	Backend Dev	POST /route endpoint
Web Dashboard (React + Recharts)	Frontend Dev	Cost & routing analytics UI
OpenTelemetry + Prometheus Integration	DevOps	Grafana dashboard ready
Feedback Loop & Model Fine-tuning Data	AI Dev	Routing accuracy improves
Documentation Site (MkDocs / Docusaurus)	All	docs.openpair.dev live

7. Web Application Architecture (Future)

ในอนาคต OpenPair จะขยายจาก CLI/Library กลายเป็น Full-Stack Web Application ที่รองรับทีมและองค์กร:

7.1 Frontend

- Framework: React 18+ + TypeScript
- UI Library: Shadcn/UI + Tailwind CSS
- Charts: Recharts สำหรับ Cost & Routing Analytics
- State Management: Zustand หรือ TanStack Query
- Features: Dashboard, Model Registry Editor, Live Router Testing, Cost Reports

7.2 Backend API

- Framework: FastAPI (Python) — ห่อหุ้ม OpenPair Core Library
- Authentication: JWT + OAuth2 (GitHub, Google SSO)
- Database: PostgreSQL สำหรับ User Data / Usage History, Redis สำหรับ Cache
- Queue: Celery + Redis สำหรับ Async Long-running Tasks
- API Documentation: Auto-generated Swagger UI จาก FastAPI

7.3 Infrastructure

- Containerization: Docker + Docker Compose (Dev), Kubernetes (Production)
- CI/CD: GitHub Actions — Test → Build → Deploy pipeline
- Cloud: AWS (ECS/EKS) หรือ GCP (Cloud Run) — Serverless-compatible
- CDN: Cloudflare สำหรับ Frontend Assets
- Monitoring: Grafana + Prometheus + Sentry

7.4 Pricing / Business Model (SaaS)

Tier	Target	Price	Key Features
Open Source	Individual Dev	Free	CLI + Library, Community Support
Starter	Small Teams	\$29/mo	Web Dashboard, 5M tokens/mo, 3 Users
Pro	Startups	\$99/mo	Unlimited Routing,

Tier	Target	Price	Key Features
			Analytics, 10 Users, SLA 99.9%
Enterprise	Large Org	Custom	On-Premise, SSO, Dedicated Support, SLA 99.99%

8. Benchmarking & Evaluation Plan

8.1 Evaluation Metrics

Metric	Target	Measurement Method
Cost Savings vs. Always-GPT4	≥ 60%	Total \$ spent over 10K test requests
Routing Accuracy	≥ 90%	Human-labeled dataset 1K prompts
Quality Score (0-1)	≥ 0.85	LLM Judge vs. GPT-4 Gold Standard
Routing Latency P99	< 5 ms	Load test: 1K concurrent requests
Cascade Improvement Rate	≥ 70%	% ที่ Cascade ช่วยให้ได้ผลดีขึ้น

8.2 Benchmark Datasets

- MMLU (Massive Multitask Language Understanding) — สำหรับวัด Reasoning Routing
- HumanEval / MBPP — สำหรับวัด Code Routing Accuracy
- MT-Bench — สำหรับวัด Multi-turn Conversation Quality
- Custom Internal Dataset — Prompt จาก Real-world Use Cases ที่รวบรวมจากผู้ใช้งาน Beta

9. Open Questions & Decisions Needed

Question	Priority	Notes / Options
Rust vs. Go สำหรับ Core Engine?	P0 — ต้องตัดสินใจก่อน Phase 1	Rust: Performance ดีกว่า / Go: เรียนรู้ง่ายกว่า
Embedding Model: BERT vs. BGE vs. Nomic?	P1	Trade-off: Size vs. Accuracy vs. Speed
กลยุทธ์ตั้งราคา SaaS?	P2	ดู Section 7.4 สำหรับ Options เบื้องต้น
รองรับ Streaming (SSE/WebSocket)?	P1	จำเป็นสำหรับ UX ดี — ต้องออกแบบ API
License: MIT vs. Apache 2.0 vs. BSL?	P2	BSL = Open แต่ Commercial Restriction

10. Document History

Version	Date	Author	Changes
1.0	May 2025	Product Team	Initial Draft — Core Architecture & MVP Scope
1.1	May 2025	AI Expert Review	Added Sections 7-9: Web App, Benchmarking, Open Questions. Expanded Tech Specs, API Design, Non-functional Requirements